Luke Peterson

3lukepeterson@gmail.com | GitHub | LinkedIn
U.S. Citizen | Kaysville, Utah

SUMMARY

Computer Science senior at Weber State University, graduating December 2026, with development experience in C#/.NET, Java, Python, TypeScript, and JavaScript. Designed and deployed two full-stack applications now running in production on Amazon Web Services and Railway, covering database design, authentication, containerized deployment, and an automated build and release pipeline. On team projects, led a five-developer Agile team and owned shared code the rest of the team built against.

EDUCATION

Weber State University, Ogden, UT **January 2023 – December 2026 (Expected)**
Bachelor of Science, Computer Science | Cumulative GPA: 3.56 / 4.00
Relevant Coursework: Data Structures & Algorithms, Operating Systems, Computer Architecture & Organization, Network Fundamentals & Design, Software Engineering I & II, Advanced Database Programming, Server-Side Web Architecture, Calculus I & II, Statistics
In Progress (Fall 2026): Computer Science Capstone, Human-Computer Interaction

TECHNICAL SKILLS

Languages: C#, Java, Python, TypeScript, JavaScript, SQL, HTML, CSS
Frameworks: .NET, ASP.NET Core, Entity Framework Core, WPF, Avalonia, JavaFX, React, Next.js, Vite, Node.js, Express, Django, Tailwind CSS
Cloud, Data & Tools: Amazon Web Services (EC2, RDS), Microsoft Azure, Railway, Docker, PostgreSQL, Prisma, Microsoft SQL Server, Git, GitHub Actions (CI/CD), Linux

WORK EXPERIENCE

Software Engineering Intern, Africompassion — africompassion.org **September 2026 – Present**
- Owned the development of a child sponsorship platform for a nonprofit school in Tanzania, turning the CEO and directors' high-level goals into concrete features by advising on what was feasible, proposing solutions, and building them in React and JavaScript so donors can select a child and fund their education.
- Designed the student and donor database in SQL and built tested endpoints into the organization's existing C# .NET API, syncing monthly donations from an external payment API so staff can see who sponsors each child, manage student profiles, and grandfather existing monthly donors in as sponsors.
- Recovered every student record and photo from the legacy site's undocumented API with a Python scraper, and updated the existing photo upload API to strip location and other metadata from every image to protect the children's privacy.

TECHNICAL PROJECTS

Important! — important-app.com **May 2026 – Present**
- Deployed a full-stack email-filtering web application and Chrome extension using TypeScript, JavaScript, React, Node.js, and Express, published on the Chrome Web Store, letting users mute their inbox and receive alerts only for the senders and keywords that matter to them.
- Designed a private, network-isolated PostgreSQL database on AWS RDS with Prisma ORM, reachable only by the containerized backend, and enforced TLS with certificate verification so user credentials are never exposed to the public internet.
- Containerized the API with Docker and deployed it to AWS EC2 behind a Caddy reverse proxy with automatic HTTPS, automating build, test, and release through a GitHub Actions CI/CD pipeline so deployments are repeatable and identical to local development.
- Integrated third-party services including Google OAuth 2.0 with a metadata-only Gmail scope, Stripe subscription billing with signature-verified webhooks, and the Anthropic Claude API for automated sender categorization.

Robot Diagnostics Dashboard — Faculty Directed Research, Team of 4 developers **August 2026 – Present**
- Developed a live diagnostics dashboard for the university's self-driving race cars in HTML, CSS, and JavaScript, gathering requirements from the course professor and keeping it framework-free and documented in architecture decision records so future students can maintain it.
- Built a responsive web and mobile interface over WebSockets that shows every system on the car at a glance and lets multiple students watch its camera and lidar or drive it from a phone in a full-screen tiling view.
- Engineered a Python ROS 2 service on the car's Linux computer that reports system health and runs a safety watchdog to stop the car if the driving connection drops, and patched a C++ driver bug that left motor temperatures reading zero.

Utah Golf Times — utahgolftimes.com **August 2026 – Present**
- Built a Next.js and React tee-time aggregator in TypeScript that combines open times from courses across Davis County into one filterable list, so golfers check one page instead of every course's booking site.

- Normalized the undocumented JSON APIs behind two booking platforms into one schema with a Node.js polling worker that validates every response with Zod and updates PostgreSQL through Prisma, checking today's times most often and backing off on days further out so the booking sites aren't overloaded.

Reservation System — Team Lead, 5 developers **March – April 2026**

- Led a team of five peers to build a reservation system using C#, ASP.NET Core MVC, SQL Server, and JavaScript, designing the database schema and role-based dashboards to manage bookings and dynamically recalculate invoices.
- Directed weekly Agile sprints using Git and Jira, deployed the application to Microsoft Azure, and folded instructor feedback in as stakeholder requirements to deliver every milestone on schedule.

Utah Mammoth Odds Predictor **April 2026**

- Built an NHL win-probability web application with Python and Django, consuming the league's public REST API to project a winner and score for each upcoming game, with server-rendered Plotly charts exposing the statistics behind every forecast.
- Designed the weighted scoring model behind each forecast, normalizing raw game statistics into categories for goaltending, special teams, possession, and discipline, then revised it after observing that it leaned too heavily on the binary win-or-loss result so a forecast reflected how a team had actually played rather than whether it happened to win.
- Cached each completed prediction in a Django ORM record keyed to the game, and aggregated injury data the official API does not expose by scraping reports with BeautifulSoup, so repeat visits render from the database instead of repeating the full chain of upstream requests.

Invoice Management System — Team of 3 developers **November – December 2025**

- Built a desktop invoice application in C# on .NET 8 using WPF and XAML data binding, structuring every feature as a window, business-logic, and data-access layer over a shared OLE DB provider so each developer could work a feature vertically without collisions.
- Owned the main invoice-editing window and authored the shared data-access class both other developers ran every query through, coordinating the three feature branches into main as each was completed.
- Reproduced and resolved a crash that occurred when line items were added to an unsaved invoice and then removed, tracing it to a price lookup returning an empty string that failed on numeric conversion, then replacing that return contract with an explicit not-found value and guarding the conversion before it ran.

SoundCloud Archiver **April – May 2026**

- Built a cross-platform desktop application in C# with Avalonia and the Model-View-ViewModel pattern to keep a local music library backed up and in sync with its online source, using async/await so downloads report progress without freezing the interface.
- Reconciled the local library against its source on every sync by diffing the track IDs returned upstream against a JSON manifest, surfacing tracks that had disappeared for the user to confirm rather than deleting anything silently.
- Represented playlist membership through filesystem symlinks so a track belonging to several playlists is stored on disk only once.

Blackjack Simulator **October 2025 – January 2026**

- Built a desktop card-game simulator in Java 21 with JavaFX, implementing full casino rules including hand splitting, doubling down, insurance, side-bet payout evaluation, and Hi-Lo card counting, then packaged it with Maven so friends could install and run it on Windows, macOS, and Linux.
- Separated the rules engine from the interface behind two observer interfaces, giving the domain model no dependency on the UI framework so game logic and presentation could each change without touching the other.

ADDITIONAL EXPERIENCE

Kitchen and Service Worker — Panda Express, Farmington, UT **September 2023 – February 2024**

30 hours per week

- Cooked menu items to order and prepared sauces during continuous peak service, keeping stations stocked and food-safety standards met under sustained time pressure.
- Ran the register, served customers, and reset the dining area, moving between kitchen and front-of-house as demand shifted.

Cashier — Arctic Circle, Kaysville, UT **June 2021 – October 2021**

20 hours per week

- Took food and drink orders and processed customer payments, serving orders at the counter, drive-through window, and tables.
- Cleaned and restocked food preparation areas, cooking surfaces, utensils, and display cases between rushes.